

16. Съставни структури от данни: речници, вектори и списъци (свързан и двусвързан списък). Примерен код на C++.

1. Въведение

Структурата от данни е начин за организиране и съхранение на данни в паметта на компютъра така, че да могат да бъдат ефективно достъпвани и изменяни. Съставните структури са такива, които съхраняват колекция от елементи и предоставят операции за добавяне, премахване, търсене и обхождане на тези елементи.

Важно е да се прави разлика между:

- Абстрактен тип данни - мат. модел, който описва какви операции се поддържат и какво правят те, но не и как се реализират.
- Структура от данни - конкретна реализация на абстрактен тип данни, която определя как данните се подреждат в паметта и как се изпълняват операциите.

2. Вектори

2.1 Абстрактен тип данни "вектор"

Векторът (или масив с динамичен размер, `dynamic array`) е последователност от елементи, всеки от които има индекс (позиция), започващ от 0.

Операция	Описание	Сложност
----------	----------	----------

<code>at(i) / operator[]</code>	връща елементите на позиция <code>i</code>	$O(1)$
<code>set(i, e)</code>	заменя елемента на позиция <code>i</code> със <code>e</code>	$O(1)$
<code>insert(i, e)</code>	вмъква <code>e</code> на позиция <code>i</code> , измествайки следващите	$O(n)$
<code>erase(i)</code>	премахва елемента на позиция <code>i</code>	$O(n)$
<code>size()</code>	брой елементи	$O(1)$
<code>empty()</code>	дали векторът е празен	$O(1)$
<code>push_back(e)</code>	добавя в края	$O(1)$

2.2 Реализация чрез непрекъснат масив

Векторът се реализира с **непрекъснат блок памет** (масив) от фиксиран капацитет (`capacity`), от който в момента са заети `size` елемента. Ключовото предимство е, че адресът на елемент с индекс `i` се изчислява пряко:

$$\text{адрес}(A[i]) = \text{базов_адрес} + i * \text{sizeof}(\text{тип})$$

затова достъпът по индекс е $O(1)$.

2.3 Растеж на капацитета

Когато при `push_back` заетите елементи достигнат капацитета, се заделя нов, по-голям масив (обикновено **двойно** по-голям), елементите се копират в него и старият се освобождава. Едно такова презаделяне струва $O(n)$, но тъй като капацитетът се удвоява, презаделяния се случват рядко.

2.4 Примерен код на C++

```
class Vector {
private:
    int* data; // указател към масив
    int sz; // размер (брой заети места)
```

```

int cap; // капацитет

void reserve(int newCap) {
    if (newCap <= cap) return ;

    int* newData = new int[newCap];
    for (int i = 0; i < sz ; i++) {
        newData[i] = data[i];
    }
    delete[] data;
    data = newData;
    cap = newCap;
}

public:
    Vector() : data(nullptr), sz(0), cap(0) {}
    ~Vector() { delete[] data; }

    int size() const {
        return sz;
    }

    bool empty() const {
        return sz == 0;
    }

    int& operator[](int i) {
        return data[i];
    }

    const int& operator[](int i) {
        return data[i];
    }

    int& at(int i) {
        if (i < 0 || i >= sz) cout << "Invalid Index" << end

```

```

1;

    return data[i];
}

void push_back(const int& e) {
    if (sz == cap) reserve(capp == 0 ? 1 : cap * 2);
    data[sz++] = e;
}

void insert(int i, const int& e) {
    if (sz == cap) reserve(cap == 0 ? 1 : 2 * cap);
    for (int j = sz; j > i; --j) {
        data[j] = data[j - 1];
    }

    data[i] = e;
    sz++;
}

void erase(int i) {
    for (int j = i; j < sz - 1; j++) {
        data[j] = data[j + 1];
    }
    sz--;
}
};

```

3. Списъци

За разлика от вектора, свързаният списък не държи елементите в непрекъсната памет. Всеки елемент се пакетира във възел (node), който освен данните пази и указател към съседен възел. Така вмъкване и премахване не изискват изместване на елементи - достатъчно е "пренасочване" на указатели.

3.1 Едносвързан списък

Всеки възел съдържа данна и указател `next` към следващия възел. Последният възел сочи към `nullptr`. Списъкът се идентифицира с указател към първия възел - `head`.



Свойства:

- Достъп до елемент с индекс `i` изисква обхождане от `head` → $O(n)$
- Вмъкване/премахване **в началото** е $O(1)$
- Не изисква непрекъснатата памет, расте
- Не може да се обхожда назад

```
class SinglyLinkedList {
private:
    struct Node {
        int value;
        Node* next;
        Node(const int& v, Node* n = nullptr) : value(v), next(n) {}
    };
    Node* head;
    int sz;

public:
    SinglyLinkedList() : head(nullptr), sz(0) {}
    ~SinglyLinkedList() {
        // освобождаване на
        ВСИЧКИ ВЪЗЛИ
        while (head) {
            Node* tmp = head;
            head = head->next;
            delete tmp;
        }
    }
};
```

```

    }
}

void push_front(const int& v) {
    head = new Node(v, head);
    SZ++;
}

void pop_front() {
    if (!head) return;
    Node* tmp = head;
    head = head->next;
    delete tmp;
    SZ--;
}

bool contains(const int& v) const {
    for (Node* p = head; p != nullptr; p = p->next)
        if (p->value == v) return true;
    return false;
}

void print() const {
    for(Node* p = head ; p!= nullptr ; p = p-> next) {
        cout << p->value << " -> ";
    }
    cout << "nullptr" << endl;
}
};

```

Вмъкване в началото нагледно:



3.2 Двусвързан списък

Всеки възел има два указателя: `next` (към следващия) и `prev` (към предходния). Това позволява обхождане в двете посоки и премахване на даден възел за $O(1)$, ако вече имаме указател към него (не е нужно търсене на предходния).

Често се използват два специални **сигнални (sentinel) възела** - `header` и `trailer` - които не носят данни, но опростяват кода, защото премахват специалните случаи в краищата.

Сравнение на свойствата:

Характеристика	Едносвързан	Двусвързан
Указатели на възел	1 (<code>next</code>)	2 (<code>next</code> , <code>prev</code>)
Памет на възел	по-малко	повече
Обхождане назад	не	да
Премахване на даден възел	$O(n)$ (нужен е предходният)	$O(1)$
Вмъкване	в началото $O(1)$	и в двата края $O(1)$

```
class DoublyLinkedList {
private:
    struct Node {
        int value;
        Node* prev;
        Node* next;
        Node(const int& v = T()) : value(v), prev(nullptr), next(nullptr) {}
    };
    Node* header;    // възел в началото
    Node* trailer;   // възел в края
    int sz;

    // Вмъкване на нов възел със стойност v ПРЕДИ възела p – O(1)
    void insertBefore(Node* p, const int& v) {
```

```

        Node* n = new Node(v);
        Node* before = p->prev;
        n->prev = before;  n->next = p;
        before->next = n;  p->prev = n;
        sz++;
    }

    // Премахване на възел p – O(1)
    void remove(Node* p) {
        Node* before = p->prev;
        Node* after  = p->next;
        before->next = after;
        after->prev  = before;
        delete p;
        sz--;
    }

public:
    DoublyLinkedList() : sz(0) {
        header = new Node();
        trailer = new Node();
        header->next = trailer;
        trailer->prev = header;
    }

    ~DoublyLinkedList() {
        while (header) {
            Node* tmp = header;
            header = header->next;
            delete tmp;
        }
    }

    bool empty() const { return sz == 0; }
    int size()  const { return sz; }

```

```

    void push_front(const int& v) { insertBefore(header->next, v); }
    void push_back(const int& v) { insertBefore(trailer, v); }
}

void pop_front() { if (!empty()) remove(header->next); }
void pop_back() { if (!empty()) remove(trailer->prev); }

// Обхождане напред
void printForward() const {
    for (Node* p = header->next; p != trailer; p = p->next)
        std::cout << p->value << " ";
    std::cout << "\n";
}

// Обхождане назад – възможно благодарение на prev
void printBackward() const {
    for (Node* p = trailer->prev; p != header; p = p->prev)
        std::cout << p->value << " ";
    std::cout << "\n";
}
};

```

4. Речници

4.1 Абстрактен тип данни “речник”

Речникът съхранява двойки (**ключ, стойност**) и позволява бързо намиране на стойността по ключ. За разлика от вектора, индексът не е целочислена позиция, а произволен ключ (може да е низ, число и т.н.).

Основни операции:

- `insert(k, v)` - добавя двойка

- `find(k)` - връща стойността, свързана с ключа k
- `erase(k)` - премахва двойката с ключ k
- `contains(k)` - проверка дали ключът съществува

Различаваме подреден речник (ключовете могат да се обхождат в наредба) и неподреден речник (без гаранция за наредба, но обикновено по-бърз).

4.2 Реализации

Може да се реализира чрез хеш таблица (`unordered_map`) - среден достъп $O(1)$ без наредба. Или чрез балансирано двоично дърво (BST) - достъп $O(\log n)$ при подредени ключове.

4.3 Примерен код на C++

```
#include <vector>
#include <list>
#include <string>
#include <functional>

template <typename K, typename V>
class HashMap {
private:
    struct Entry { K key; V value; };
    std::vector<std::list<Entry>> buckets; // масив от кофи
    (списъци)
    int count;

    int indexFor(const K& key) const {
        return std::hash<K>{}(key) % buckets.size();
    }

public:
    HashMap(int n = 16) : buckets(n), count(0) {}

    // Въмъкване или обновяване – среден O(1)
```

```

void insert(const K& key, const V& value) {
    auto& chain = buckets[indexFor(key)];
    for (auto& e : chain)
        if (e.key == key) { e.value = value; return; }
    chain.push_back({key, value});
    ++count;
}

// Търсене – среден O(1)
bool find(const K& key, V& out) const {
    const auto& chain = buckets[indexFor(key)];
    for (const auto& e : chain)
        if (e.key == key) { out = e.value; return true; }
    return false;
}

// Премахване – среден O(1)
void erase(const K& key) {
    auto& chain = buckets[indexFor(key)];
    for (auto it = chain.begin(); it != chain.end(); ++i
t)
        if (it->key == key) { chain.erase(it); --count; r
eturn; }
}

int size() const { return count; }
};

```

5. Сравнение на сложности

Таблицата дава типичните (средни) сложности на основните операции:

Операция	<code>vector</code>	едносвързан списък	двусвързан списък	<code>map</code> (BST)
Достъп по индекс/ключ	$O(1)$	$O(n)$	$O(n)$	$O(\log n)$
Вмъкване в началото	$O(n)$	$O(1)$	$O(1)$	$O(\log n)$
Вмъкване в края	$O(1)$ *	$O(n)$ **	$O(1)$	$O(\log n)$
Премахване на даден възел	$O(n)$	$O(n)$	$O(1)$	$O(\log n)$
Търсене на стойност	$O(n)$	$O(n)$	$O(n)$	$O(\log n)$
Подредено обхождане	да	да	да	да
Памет (overhead)	малък	средно (1 указ./ възел)	по-голям (2 указ./ възел)	голям

Правила за избор:

- Чест произволен достъп и добавяне в края → **вектор**
- Чести вмъквания/премахвания в краищата или в средата (при наличен указател) → **двусвързан списък**
- Бързо съпоставяне ключ → стойност без наредба → `unordered_map` (хеш-таблица)
- Същото, но с нужда от подредени ключове или диапазонни заявки → `map` (BST)